AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science

Faculty of Science and Technology

CSC 2210: OBJECT ORIENTED PROGRAMMING 2

Summer 2025-2026

Section : S

Group No: 7

Project Report On

Stationery Shop Management System

Supervised By

Md. Sadik Tasrif Anubhove

Submitted By

Name	ID
1. Hasnine Alom Rimon	24-57079-1
2. Tousif Bin Mostofa	24-58475-2
3. Diba Biswas	24-60003-3
4. Jannatul Ferdous Mim	24-59940-3
5. Kazi Simi Ashnah	24-58688-2

Obtained Marks for CO2 and CO4

Assessment Criteria	Not Attended (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)	Total	Signature
CO2: Requirement fulfillment						/ 5	
CO2: Validation & Verification						/ 5	
CO4: Design & Innovation						/ 5	
CO4: Teamwork & Communication						/ 5	

Table of Contents

Section / Chapter Title	Page
1. Introduction	**3**
2. Feature List / User Story	**3**
2.1 Shop Operator / Administrator Role	3
2.2 CRUD Operations Matrix	4
3. Working Principle & Development Process	**4**
3.1 Tools & Technologies	4
3.2 Interface Design Steps	4
3.3 Database Connection	7
4. Methodology	**8**
5. ER Diagram	**8**
5.1 Normalization (UNF to 3NF)	9
5.2 Finalization & Schema	10
6. UML Diagram	**11**
6.1 Use Case Diagram	11
6.2 Activity Diagram	11
7. SQL Queries & Screenshots (Insert / Select / Update / Delete)	**12**
7.1 Insert Operation	12
7.2 Select / View Operation	13
7.3 Update Operation	14
7.4 Delete Operation	15
8. Marketing Purpose / Cost Implementation	**16**
8.1 Market Need	16
8.2 Target Customers	16
8.3 Estimated Cost Implementation	16
9. Conclusion	**17**
9.1 Summary & Key Learnings	17
9.2 Future Improvements	17
10. Reference	**18**

1. Introduction

Stationery shops play an essential role in schools, colleges, universities, and offices by providing everyday study and work materials such as pens, paper, notebooks, geometry boxes, and art supplies. In most traditional stationery stores, shopkeepers record product stocks, purchase costs, and sales manually using physical notebooks or paper receipts. While this manual method is simple to start with, it creates several serious problems as the number of items and daily customer visits increase.

In a manual system, the shopkeeper cannot easily know when an item is running out of stock. For example, high-demand items like ball pens or A4 paper reams might suddenly become empty during busy exam hours, resulting in lost sales and customer dissatisfaction. Finding supplier contact details when placing a new order also requires flipping through old receipt piles. Furthermore, manual arithmetic calculations often lead to pricing and stock balance mistakes. Writing down every stock entry by hand takes significant time that could otherwise be spent attending to customers.

The main purpose of the Stationery Shop Management System is to solve these everyday problems through a straightforward desktop application. Developed in C# Windows Forms with Microsoft SQL Server Express, the application enables store operators to record stationery categories, manage supplier contact profiles, keep track of individual products, update unit prices and quantities, and receive automatic alerts whenever inventory drops below a safety reorder level.

A desktop-based application is particularly well-suited for small and medium retail stationery shops. Unlike web-based solutions that require a continuous internet connection and monthly cloud server payments, a Windows desktop application operates locally on the store computer. It offers instant response times, complete privacy of business records, and zero recurring hosting costs. The software is designed for a single shop operator or store manager, ensuring that anyone with basic computer knowledge can navigate the system, update stock, and find product details without specialized training.

2. Feature List / User Story

The Stationery Shop Management System is structured around a single operational role: the Shop Operator / Administrator. This accurately reflects the actual implementation of the application, where the store manager or cashier handles all daily store inventory actions from one unified desktop environment.

2.1 Shop Operator / Administrator Role

The Shop Operator can perform the following key actions within the system:

- **Application Login:** The operator enters login credentials on the initial login screen (Form1) to unlock access to store management forms.
- **Real-Time Dashboard Overview:** Upon login, the Main Dashboard displays live counts of Total Products, Total Categories, and Total Suppliers, alongside a specialized DataGridView listing all items whose current quantity is at or below the reorder threshold.
- **Stationery Category Management:** The user can create new product categories (e.g., Writing, Paper, School Supplies, Art Supplies), browse existing categories, modify category names, and remove unused categories.
- **Supplier Relationship Management:** The user can record new supplier profiles including company name, phone number, email address, and physical address, as well as update vendor contacts and view the supplier directory.
- **Product Inventory Management:** The user can add new stationery items by specifying product name, assigning category and supplier via dropdown selectors, and entering unit price, stock quantity, and safety reorder level.
- **Dynamic Product Search:** The operator can filter the product inventory in real time by typing product name keywords into the search box, eliminating manual scrolling through large tables.

- **Low-Stock Visual Status:** The product grid dynamically marks vulnerable products with a 'Low Stock' badge, giving immediate visual warning during store replenishment.
- **Application Exit Confirmation:** A confirmation dialog verifies whether the operator genuinely wishes to terminate the program before shutting down.

2.2 CRUD Operations Matrix

The following table details the Create, Read, Update, and Delete capabilities implemented for each core entity in the application:

Entity / Module	Create (Add)	Read (View)	Update (Edit)	Delete (Remove)
Categories	Add new category name with validation	View sorted category list in DataGridView	Modify category name by selecting row	Delete category with foreign key constraint check
Suppliers	Add supplier name, phone, email, address	View all suppliers with contact info in DataGridView	Edit supplier phone, email, or address	Delete supplier with linked products check
Products	Add product with category, supplier, price, qty, reorder	View joined product catalog with status and search	Update price, stock quantity, reorder level, and links	Delete selected product from database

3. Working Principle & Development Process

Building the Stationery Shop Management System followed a structured development approach: configuring the development environment, creating the SQL Server relational database, building business entity models, implementing user interface forms in Visual Studio, and connecting user controls to database tables using ADO.NET.

3.1 Tools & Technologies

The project was developed and tested using standard Microsoft technologies covered in the OOP 2 curriculum:

- **Device / Operating System:** Windows 11 (64-bit).
- **Integrated Development Environment (IDE):** Microsoft Visual Studio 2022 Community Edition.
- **Programming Language:** C# (.NET Framework 4.8).
- **User Interface Framework:** Windows Forms (WinForms).
- **Database Management System (DBMS):** Microsoft SQL Server Express (.\SQLEXPRESS).
- **Database Administration Tool:** SQL Server Management Studio (SSMS).
- **Database Connectivity:** ADO.NET using System.Data.SqlClient library.

3.2 Interface Design Steps

The development of the application interface proceeded through the following sequential steps, with real application screenshots illustrating each milestone:

Step 1: Creating the C# Windows Forms Project & Login Interface

We initialized a new Windows Forms App (.NET Framework 4.8) project named 'StationeryShopManagementSystem' in Visual Studio. We created Form1 as the system entry point with text boxes for username and password, a 'Log in' button, and an 'Exit' button. The code verifies that neither field is empty and performs basic username and password validation against preset credentials

('admin' / '123'). This is a simple validation check to ensure only the shop operator enters the system. When successful, an information dialog appears, Form1 hides itself, and the Main Dashboard opens.

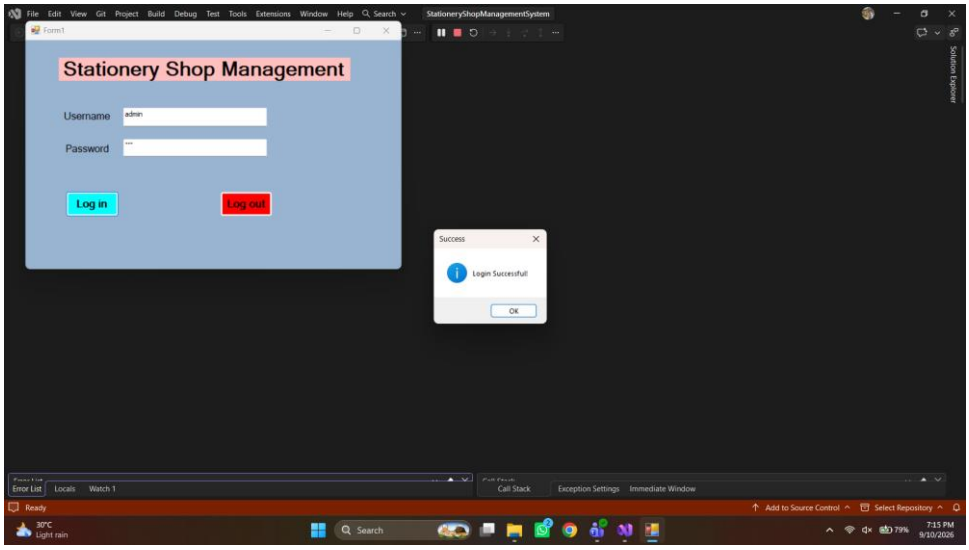


Fig 1: *Login Form (Form1) with login dialog*

Step 2: Designing the Main Dashboard Interface (MainForm)

The Main Dashboard acts as the central control hub. It features two primary sections: a navigation panel on the left containing buttons for Products, Categories, Suppliers, and Exit; and an analytical summary panel on the right. During form load, the system executes aggregate scalar queries to display total counts of products, categories, and suppliers. It also populates a specialized DataGridView showing all products whose QuantityInStock is less than or equal to their ReorderLevel.

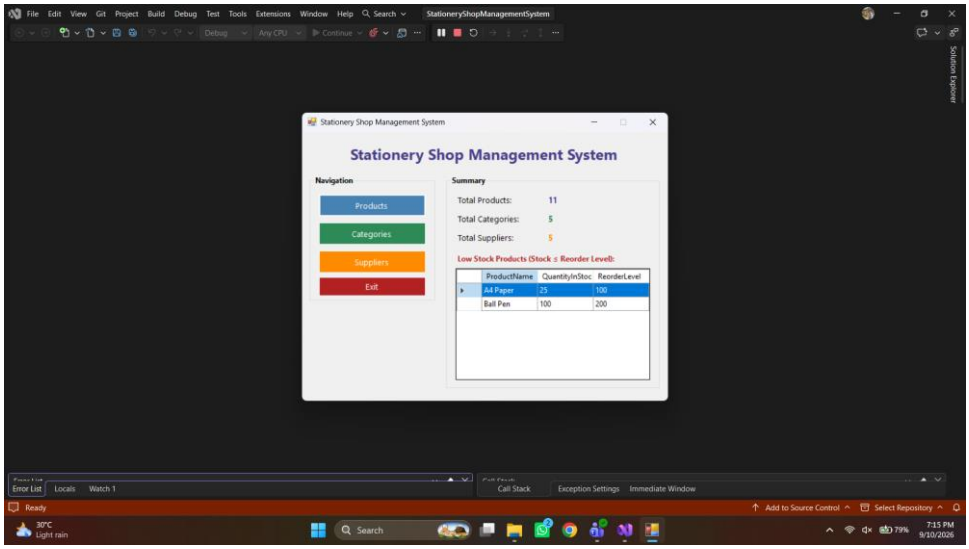


Fig 2: *Main Dashboard (MainForm) displaying live counts and low-stock grid*

Step 3: Creating the Category Management Form (CategoryForm)

CategoryForm allows the operator to organize stationery items into logical groupings such as Writing, Paper, School Supplies, and Office Supplies. The form contains an ID text box (read-only), a Category Name input text box, and buttons for Add, Update, Delete, and Clear. Clicking on any row in the DataGridView automatically populates the text box for editing. The form checks for empty inputs and catches SQL foreign key violation errors if the user attempts to delete a category linked to existing products.

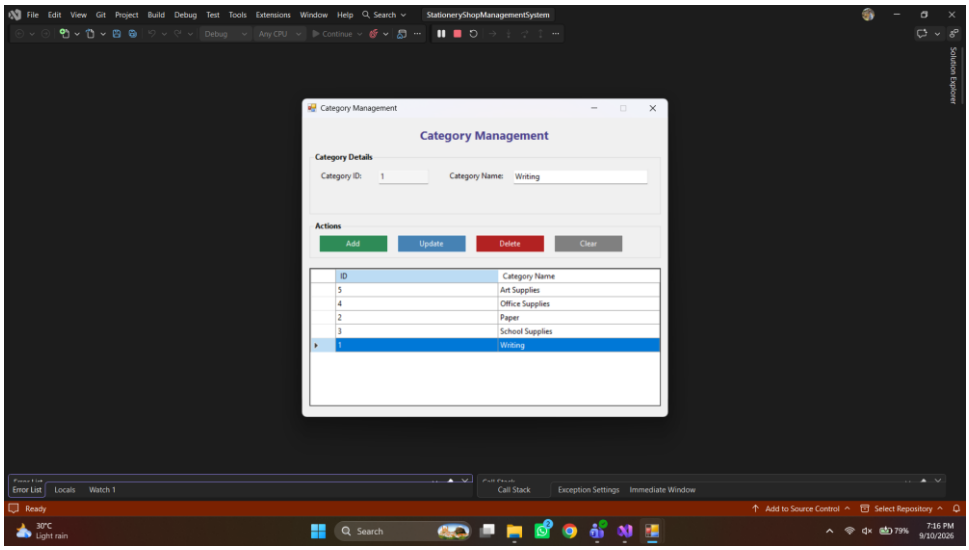


Fig 3: *Category Management Form (CategoryForm) with CRUD operations and DataGridView*

Step 4: Creating the Supplier Management Form (SupplierForm)

SupplierForm records vendor details so the store can restock items promptly. It includes input controls for Supplier ID (read-only), Supplier Name, Phone Number, Email Address, and Physical Address. Validation checks ensure that supplier name is not blank, phone numbers contain no letters, and email addresses adhere to standard email syntax via regular expressions. The DataGridView presents all suppliers ordered alphabetically.

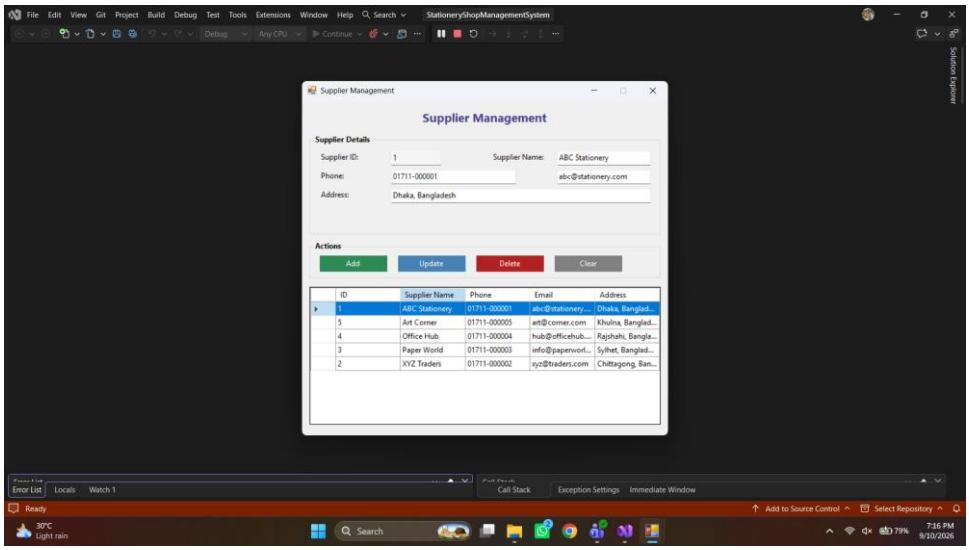


Fig 4: *Supplier Management Form (SupplierForm) with contact fields and validation*

Step 5: Implementing the Product Management Form (ProductForm)

ProductForm is the most comprehensive form in the application. It includes inputs for Product ID, Product Name, Category (populated dynamically via ComboBox), Supplier (populated via ComboBox), Unit Price, Quantity in Stock, and Reorder Level. A live search bar allows searching by product name. The DataGridView displays joined table results showing category and supplier names, along with a computed 'Low Stock' alert column.

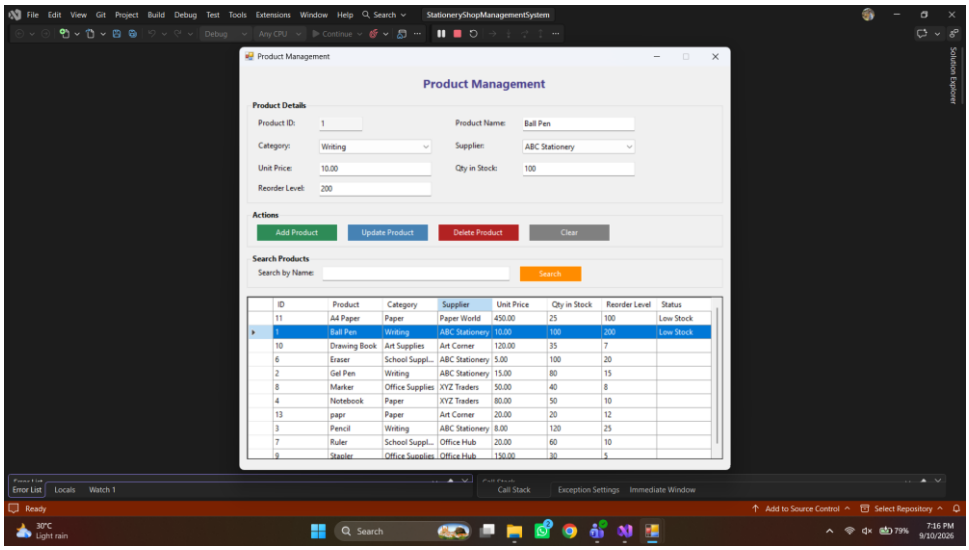


Fig 5: *Product Management Form (ProductForm) with search, ComboBox bindings, and stock status*

Step 6: Implementing Application Exit Confirmation

To prevent accidental application shutdown and unsaved work, the Exit button on the Main Dashboard triggers a confirmation dialog prompting the user 'Are you sure you want to exit the application?'. Clicking 'Yes' cleanly exits the application environment.

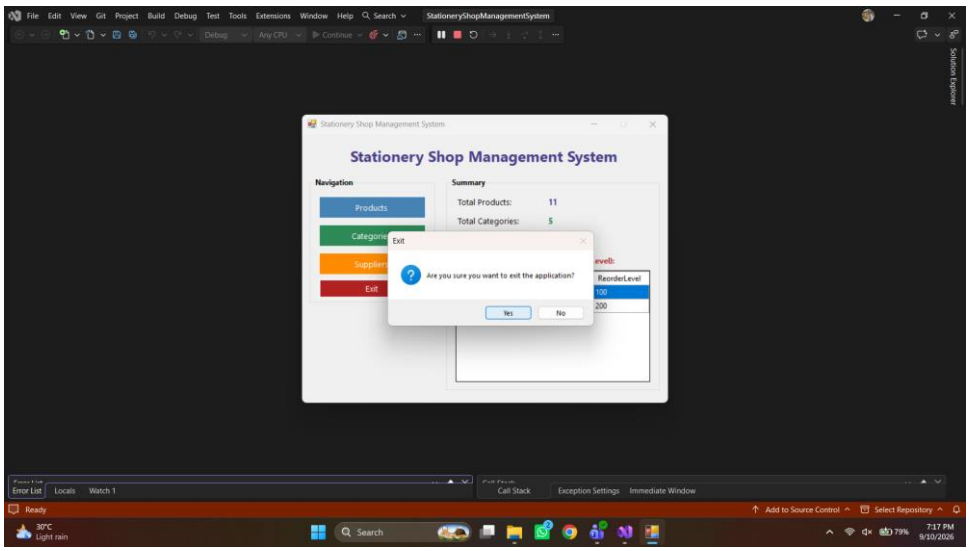


Fig 6: *Exit Confirmation Dialog ensuring safe program shutdown*

3.3 Database Connection

Database communication is centralized inside a dedicated static helper class named DatabaseHelper located in the Helpers folder. This design pattern ensures that the connection string is written in exactly one location, making maintenance simple if the SQL Server instance name changes.

The helper class configures a standard connection string targeting Microsoft SQL Server Express using Windows Integrated Security:

```
private static readonly string ConnectionString =
    @"Server=.\SQLEXPRESS;Database=StationeryShopDB;Integrated Security=True;";

public static SqlConnection GetConnection()
{
    SqlConnection connection = new SqlConnection(ConnectionString);
    connection.Open();
    return connection;
}
```

When opening a connection, ADO.NET classes work together as follows:

- **SqlConnection:** Establishes a physical session between the C# application process and the SQL Server Express database engine.
- **SqlCommand:** Carries parameterized Transact-SQL statements (INSERT, UPDATE, DELETE, SELECT) to the server for execution.
- **SqlDataAdapter & DataTable:** Fills in-memory tables from SELECT queries to bind directly as the DataSource for DataGridView controls.
- **SqlDataReader:** Provides a forward-only stream of database records used to populate Category and Supplier ComboBox dropdowns.

Below is the configuration interface for connecting the desktop project to the local SQL Server database:

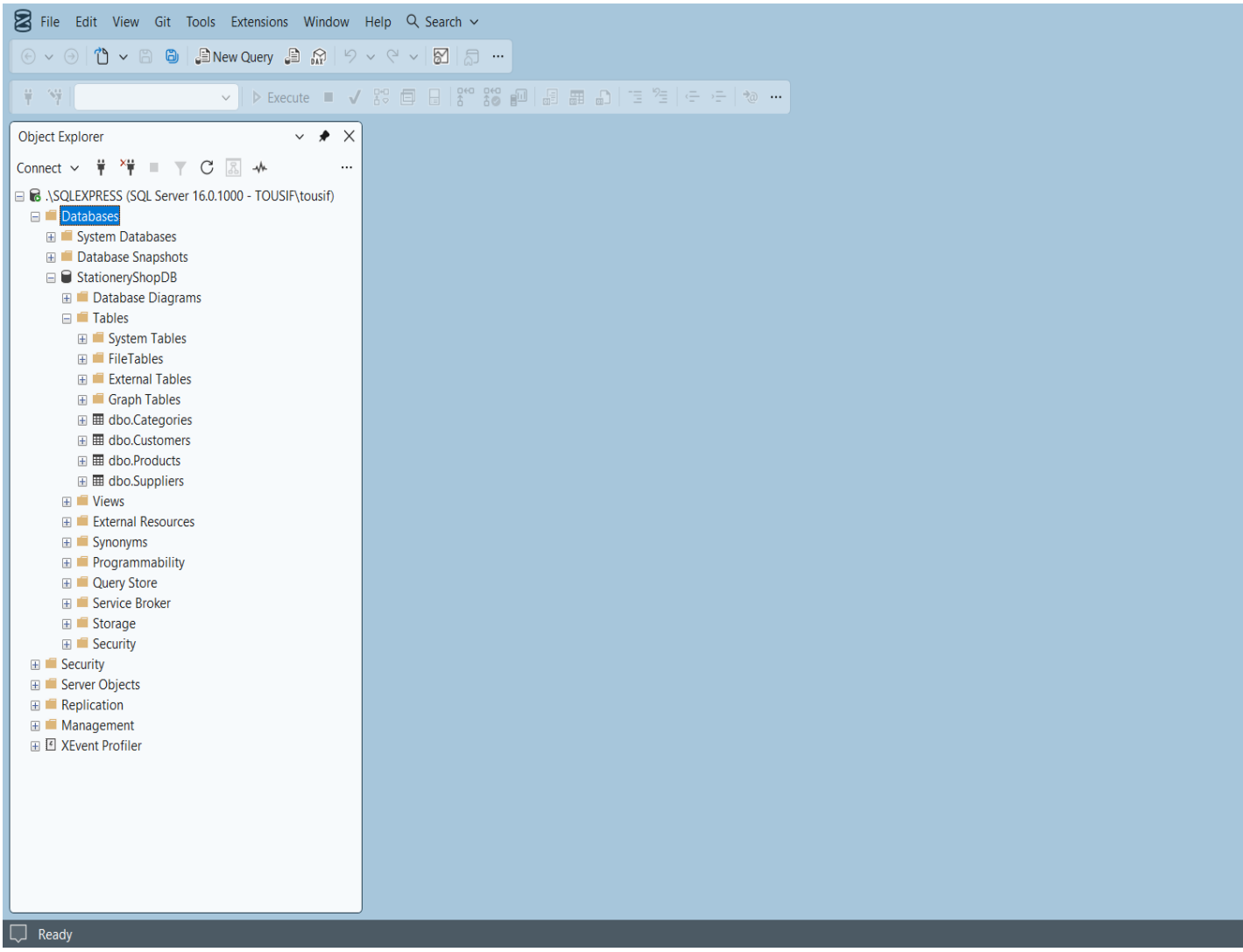


Fig 7: *Connecting C# Application to SQL Server Express (SSMS & Server Explorer)*

4. Methodology

The application uses Windows Forms for the user interface and Microsoft SQL Server for storing data. ADO.NET is used to communicate between the application and the database. Business models (Category, Supplier, Product) are used to hold and pass data between the user interface and the database, which helps keep the project organized into simple object-oriented classes.

In this project we used C# (.NET Framework 4.8) and Microsoft SQL Server Express. We created 5 Windows forms to build this project:

- **Form1 (Login Form):** Provides basic username and password validation for the store operator before opening the main interface.
- **MainForm (Dashboard):** Central navigation window displaying analytical summary counters and real-time low-stock inventory alerts.
- **CategoryForm (Category Management):** Dedicated form for categorizing stationery items and maintaining the Categories table.
- **SupplierForm (Supplier Management):** Form for managing vendor contacts, phone numbers, and addresses.
- **ProductForm (Product Management):** Primary inventory window enabling full product CRUD, category/supplier dropdown linking, search filtering, and stock status tracking.

The operational data flow follows this sequence: The operator interacts with form controls (buttons, text boxes, combo boxes) -> Event handlers execute input validation -> Business model objects are populated -> DatabaseHelper provides an open SqlConnection -> A parameterized SqlCommand or SqlDataAdapter executes against StationeryShopDB -> Changes persist in SQL Server tables -> DataGridView controls refresh to display updated records -> When the user returns to MainForm, dashboard counts automatically refresh.

Project Video Link: https://drive.google.com/file/d/1x_lmd-dsnS6M0ObLZnd2n3BKdxf6DIoD/view?usp=sharing

5. ER Diagram

The Entity-Relationship (ER) diagram represents the logical design of the StationeryShopDB database. The database contains three core entities: Categories, Suppliers, and Products. Their business relationships reflect real stationery retail store operations:

- **Categories Entity:** Stores category groupings. Each category has a unique primary key CategoryID and a required CategoryName (e.g., Writing, Paper, Art Supplies).
- **Suppliers Entity:** Stores vendor contact records. Each supplier has a unique primary key SupplierID, a SupplierName, Phone, Email, and Address.
- **Products Entity:** Stores individual stationery merchandise. Each product has a primary key ProductID, ProductName, foreign key CategoryID referencing Categories, foreign key SupplierID referencing Suppliers, UnitPrice, QuantityInStock, and ReorderLevel.

The relationships between entities are as follows: One category can contain many products (1:N), but each product belongs to exactly one category. Similarly, one supplier can provide many products (1:N), but each product record is supplied by a designated supplier. These relationships are enforced in SQL Server using foreign key constraints (FK_Products_Categories and FK_Products_Suppliers).

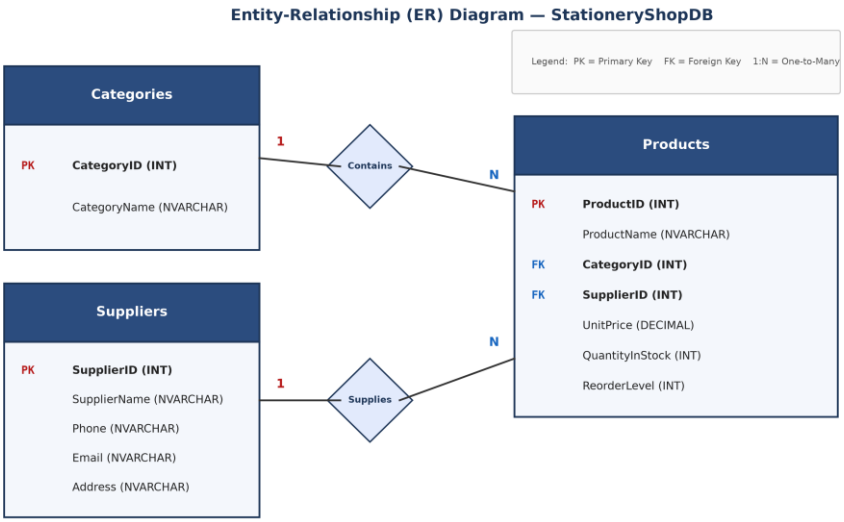


Fig 8: *Entity-Relationship (ER) Diagram for StationeryShopDB*

5.1 Normalization

Normalization is the systematic process of organizing data in a relational database to minimize redundancy and eliminate data anomalies during INSERT, UPDATE, and DELETE operations. Below is the step-by-step normalization process (UNF -> 1NF -> 2NF -> 3NF) for the stationery shop database:

Relation 1: Categories

UNF (Unnormalized Form): Raw stationery records listing categories with potential repeating groups:
Categories_UNF (CategoryID, CategoryName, {Product1, Product2, Product3...})

1NF (First Normal Form): Repeating groups of products are removed. Every attribute contains only atomic (indivisible) values, and CategoryID uniquely identifies each row:
Categories_1NF (CategoryID [PK], CategoryName)

2NF (Second Normal Form): The table has a single-attribute primary key (CategoryID). Therefore, no partial functional dependency on a composite key can exist. The relation is already in 2NF:
Categories_2NF (CategoryID [PK], CategoryName)

3NF (Third Normal Form): CategoryName depends directly on the primary key CategoryID. There are no transitive dependencies (no non-key attribute determining another non-key attribute). The relation is in 3NF:
Categories (CategoryID [PK], CategoryName)

Relation 2: Suppliers

UNF (Unnormalized Form): Vendor information recorded with multiple phone numbers or repeating supplied products:
Suppliers_UNF (SupplierID, SupplierName, Phone, Email, Address, {ProductList})

1NF (First Normal Form): Repeating product lists are removed. Each column holds atomic values, with SupplierID designated as the primary key:
Suppliers_1NF (SupplierID [PK], SupplierName, Phone, Email, Address)

2NF (Second Normal Form): The primary key consists of a single column (SupplierID). Since there is no composite key, no partial dependencies exist. All non-key attributes fully depend on SupplierID:
Suppliers_2NF (SupplierID [PK], SupplierName, Phone, Email, Address)

3NF (Third Normal Form): Attributes SupplierName, Phone, Email, and Address depend solely on SupplierID. None of the contact fields determine each other transitively. The relation is in 3NF:

Suppliers (SupplierID [PK], SupplierName, Phone, Email, Address)

Relation 3: Products

UNF (Unnormalized Form): A single flat inventory spreadsheet containing product info alongside full category names and supplier details:
StoreInventory_UNF (ProductID, ProductName, CategoryName, SupplierName, SupplierPhone, SupplierEmail, SupplierAddress, UnitPrice, QuantityInStock, ReorderLevel)

1NF (First Normal Form): All multi-valued entries are resolved into atomic columns, with ProductID as primary key:
StoreInventory_1NF (ProductID [PK], ProductName, CategoryName, SupplierName, SupplierPhone, SupplierEmail, SupplierAddress, UnitPrice, QuantityInStock, ReorderLevel)

2NF (Second Normal Form): Primary key is ProductID (single attribute). Full functional dependency exists for all columns upon ProductID; no partial key dependencies exist:
StoreInventory_2NF (ProductID [PK], ProductName, CategoryName, SupplierName, SupplierPhone, SupplierEmail, SupplierAddress, UnitPrice, QuantityInStock, ReorderLevel)

3NF (Third Normal Form): In StoreInventory_2NF, transitive dependencies exist: ProductID -> SupplierID -> (SupplierName, Phone, Email, Address) and ProductID -> CategoryID -> CategoryName. To satisfy 3NF, these transitive non-key attributes are moved to their respective parent tables (Categories and Suppliers), leaving only foreign keys in Products:

Products (ProductID [PK], ProductName, CategoryID [FK], SupplierID [FK], UnitPrice, QuantityInStock, ReorderLevel)

5.2 Finalization

The final database schema for StationeryShopDB contains the three normalized tables defined below:

Table Name	Primary Key (PK)	Foreign Keys (FK)	Non-Key Attributes
Categories	CategoryID (INT IDENTITY)	None	CategoryName (NVARCHAR(100))
Suppliers	SupplierID (INT IDENTITY)	None	SupplierName, Phone, Email, Address
Products	ProductID (INT IDENTITY)	CategoryID (FK -> Categories) SupplierID (FK -> Suppliers)	ProductName, UnitPrice, QuantityInStock, ReorderLevel

6. UML Diagram

Unified Modeling Language (UML) diagrams provide visual representations of the system's functional architecture and runtime behavior. This section presents the Use Case Diagram and the Activity Diagram for the Stationery Shop Management System.

6.1 Use Case Diagram

The Use Case diagram depicts system capabilities from the perspective of the user. In our application, there is a single actor: the Shop Operator / Administrator. The operator interacts with eight primary use cases within the system boundary:

- **Login to System:** Entry login requiring basic username and password validation.
- **View Dashboard & Summary:** Viewing store summary statistics (total products, categories, suppliers).
- **View Low Stock Alerts:** Included inside dashboard viewing to inspect items needing replenishment.

- **Manage Categories:** Adding, editing, viewing, and deleting stationery categories.
- **Manage Suppliers:** Recording vendor contact details and maintaining supplier relationships.
- **Manage Products:** Handling product catalog additions, price changes, and stock updates.
- **Search Products:** Included as a specialized capability within product management to filter items by name.
- **Exit Application:** Safely closing the program via confirmation dialog.

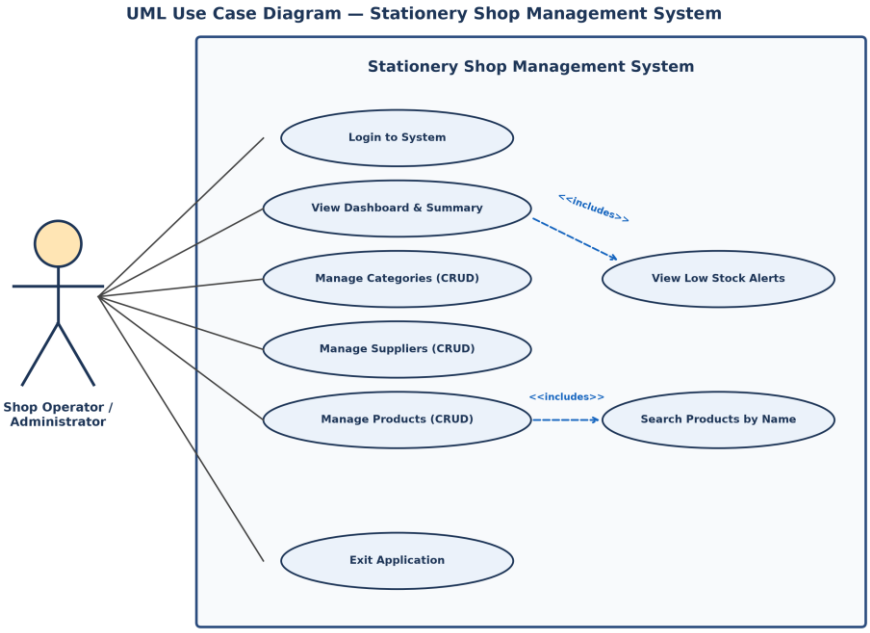


Fig 9: *UML Use Case Diagram — Stationery Shop Management System*

6.2 Activity Diagram

The Activity diagram illustrates the sequential runtime workflow of a typical user session from program startup to shutdown:

The workflow begins at the initial node when the operator launches the program. Form1 displays the login interface. The user inputs their username and password and clicks 'Log in'. The system evaluates credentials through a decision node. If invalid, an error dialog appears and the user returns to the login prompt. If valid, the Main Dashboard opens, automatically executing scalar queries to load inventory summary metrics and the low-stock grid.

From the Dashboard, the operator chooses an action branch: open Category Management, Supplier Management, Product Management, or Exit. Upon opening a management module, the user performs a CRUD or search operation. Client-side validation checks that required fields are provided and numeric values are positive. If valid, ADO.NET executes a parameterized SQL command against SQL Server. The DataGridView refreshes immediately with updated records, and returning to the dashboard synchronizes total counters. When the operator clicks Exit, a confirmation dialog verifies the request before terminating at the final activity node.

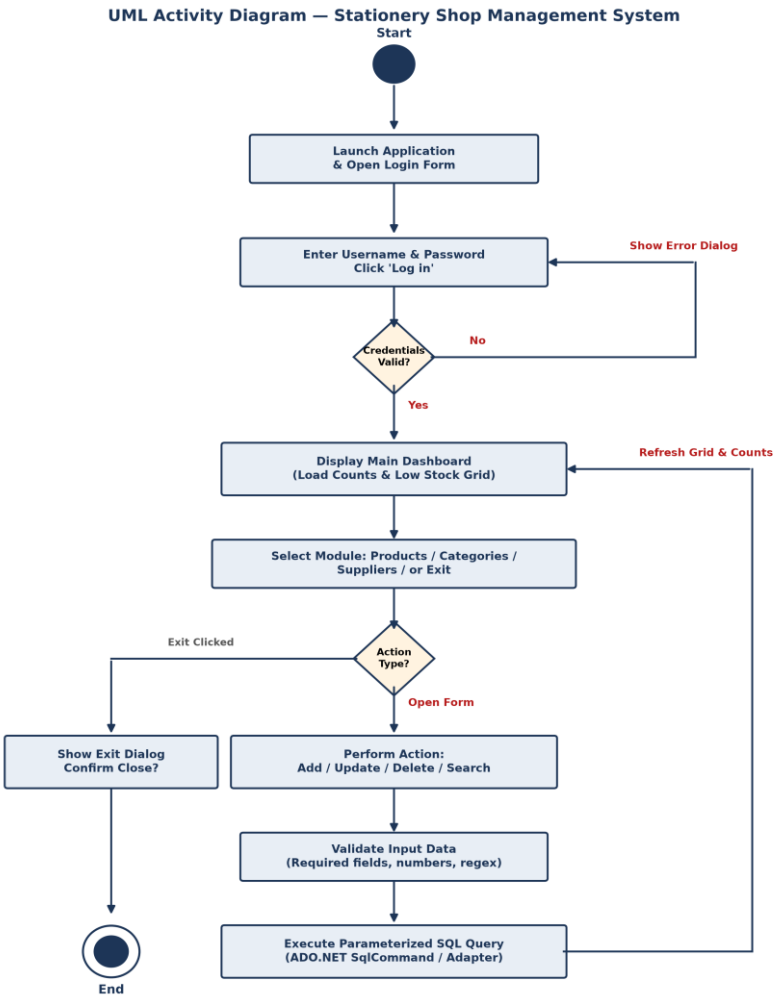


Fig 10: *UML Activity Diagram — Complete User Workflow*

7. SQL Queries & Screenshots (Insert / Update / Delete / Select)

This chapter documents the actual SQL queries and C# ADO.NET implementation logic used in the Stationery Shop Management System. All database queries use parameterized SQL commands (SqlParameter / @param placeholders) to prevent SQL injection vulnerabilities and handle special characters reliably.

7.1 Insert Operation

Adding a new category or product involves reading user inputs from text boxes, validating the values, constructing a parameterized INSERT statement, and executing it with ExecuteNonQuery().

Actual C# code for inserting a Category in CategoryForm.cs:

```
// Category Insert Logic
Category newCategory = new Category();
newCategory.CategoryName = txtCategoryName.Text.Trim();

using (SqlConnection connection = DatabaseHelper.GetConnection())
{
    string query = "INSERT INTO Categories (CategoryName) VALUES (@CategoryName)";
    SqlCommand command = new SqlCommand(query, connection);
    command.Parameters.AddWithValue("@CategoryName", newCategory.CategoryName);
    command.ExecuteNonQuery();
}
```

Actual C# code for inserting a Product in ProductForm.cs:

```
// Product Insert Logic
string query = @"INSERT INTO Products (ProductName, CategoryID, SupplierID,
                                       UnitPrice, QuantityInStock, ReorderLevel)
                 VALUES (@ProductName, @CategoryID, @SupplierID,
                         @UnitPrice, @QuantityInStock, @ReorderLevel)";

SqlCommand command = new SqlCommand(query, connection);
command.Parameters.AddWithValue("@ProductName",     newProduct.ProductName);
command.Parameters.AddWithValue("@CategoryID",      newProduct.CategoryID);
command.Parameters.AddWithValue("@SupplierID",      newProduct.SupplierID);
command.Parameters.AddWithValue("@UnitPrice",       newProduct.UnitPrice);
command.Parameters.AddWithValue("@QuantityInStock", newProduct.QuantityInStock);
command.Parameters.AddWithValue("@ReorderLevel",    newProduct.ReorderLevel);
command.ExecuteNonQuery();
```

Screenshot evidence of successful data insertion:

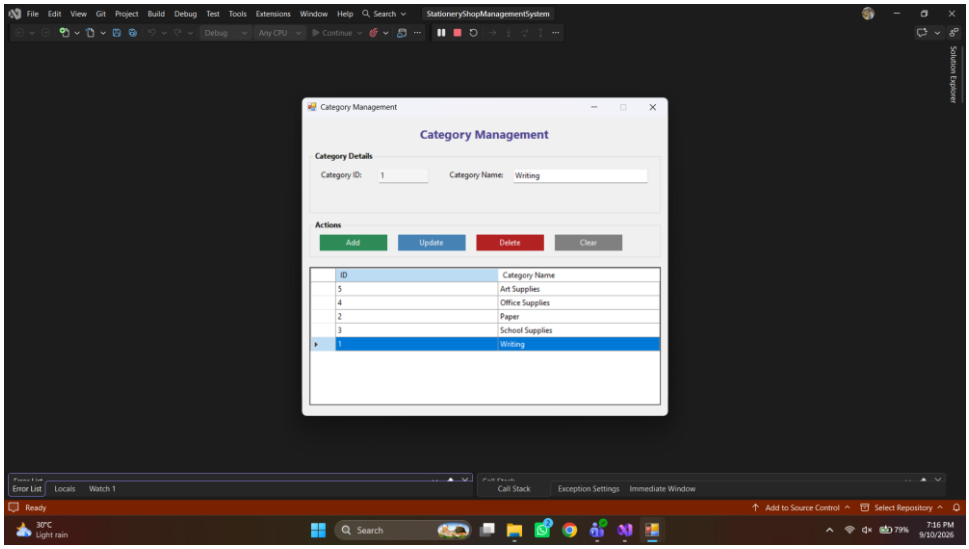


Fig 11: *Insert Operation Result — Category 'Writing' added and displayed in grid*

7.2 Select / View Operation

The application uses SELECT statements to retrieve data for DataGridView controls, ComboBox dropdowns, and dashboard metric cards. The product catalog query uses INNER JOIN to display human-readable category and supplier names instead of raw numeric IDs, and evaluates stock status dynamically with a CASE statement.

Actual Product Catalog SELECT query with INNER JOIN in ProductForm.cs:

```
SELECT p.ProductID,
       p.ProductName,
       c.CategoryName,
       s.SupplierName,
       p.UnitPrice,
       p.QuantityInStock,
       p.ReorderLevel,
       CASE WHEN p.QuantityInStock <= p.ReorderLevel THEN 'Low Stock' ELSE '' END AS StockStatus
FROM   Products p
INNER  JOIN Categories c ON p.CategoryID = c.CategoryID
INNER  JOIN Suppliers  s ON p.SupplierID = s.SupplierID
WHERE  p.ProductName LIKE @SearchTerm
ORDER  BY p.ProductName;
```

Actual Low-Stock SELECT query executed on MainForm dashboard:

```
SELECT p.ProductName, p.QuantityInStock, p.ReorderLevel
FROM   Products p
WHERE  p.QuantityInStock <= p.ReorderLevel
ORDER  BY p.ProductName;
```

Screenshot evidence of SELECT query populating the Product DataGridView:

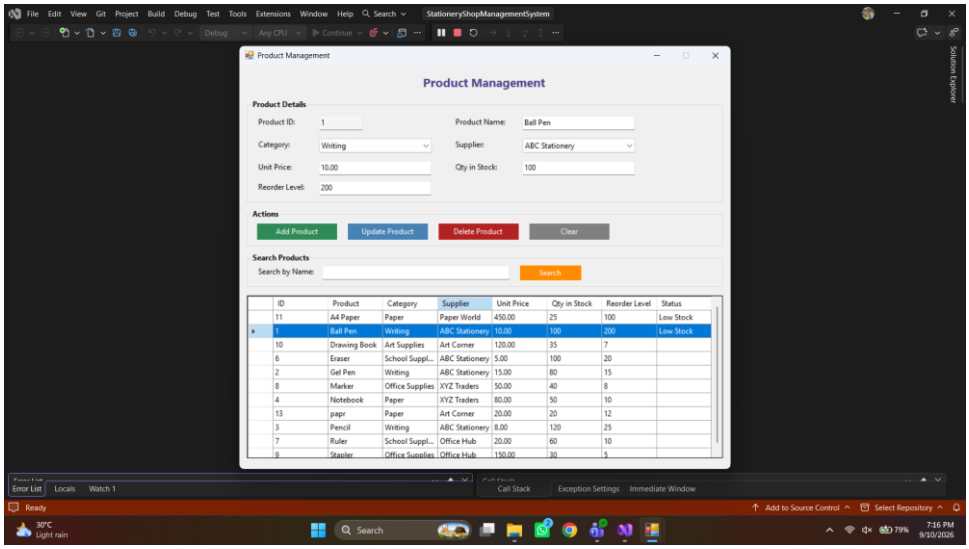


Fig 12: *Select / View Operation Result — Joined product catalog showing Category, Supplier, and Stock Status*

7.3 Update Operation

To update an existing record, the user first clicks the desired row in the DataGridView. The CellClick event extracts the primary key into a private variable (_selectedCategoryId, _selectedSupplierId, or _selectedProductId) and loads existing values into the input controls. After editing, the user clicks Update, triggering a parameterized UPDATE query.

Actual C# code for updating a Supplier in SupplierForm.cs:

```
string query = @"UPDATE Suppliers
                 SET SupplierName = @SupplierName,
                     Phone        = @Phone,
                     Email        = @Email,
                     Address      = @Address
                 WHERE SupplierID = @SupplierID";

SqlCommand command = new SqlCommand(query, connection);
command.Parameters.AddWithValue("@SupplierName", txtSupplierName.Text.Trim());
command.Parameters.AddWithValue("@Phone",        txtPhone.Text.Trim());
command.Parameters.AddWithValue("@Email",        txtEmail.Text.Trim());
command.Parameters.AddWithValue("@Address",      txtAddress.Text.Trim());
command.Parameters.AddWithValue("@SupplierID",   _selectedSupplierId);
command.ExecuteNonQuery();
```

Screenshot evidence of selecting a record for update:

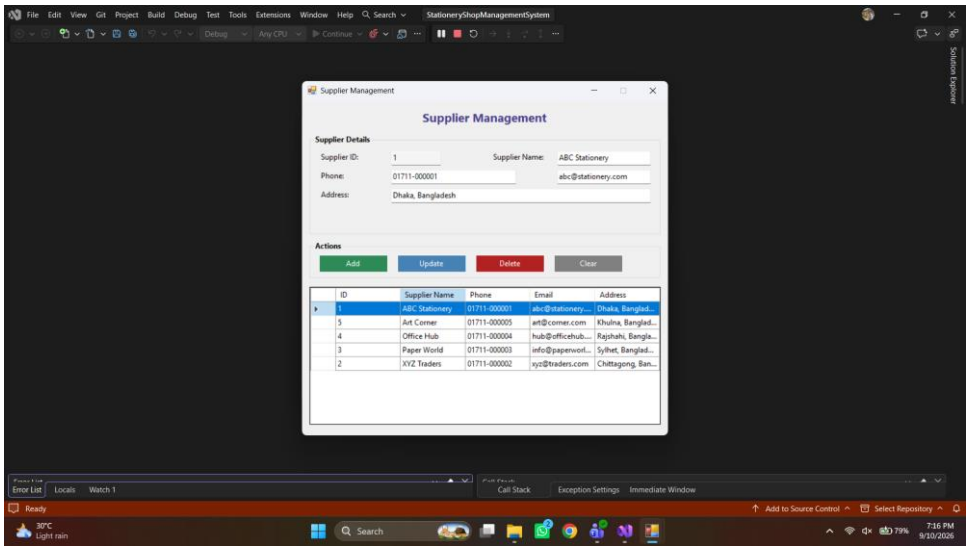


Fig 13: *Update Operation — Supplier 'ABC Stationery' loaded into input controls for editing*

7.4 Delete Operation

Deleting a record requires selecting the item, clicking Delete, and confirming the prompt dialog. The application incorporates essential foreign key protection: if an operator attempts to delete a category or supplier that is currently referenced by products in the Products table, the database raises SqlException error number 547. The code catches this error specifically and informs the user rather than crashing.

Actual C# code for deleting a Category with FK protection in CategoryForm.cs:

```
try
{
    using (SqlConnection connection = DatabaseHelper.GetConnection())
    {
        string query = "DELETE FROM Categories WHERE CategoryID = @CategoryID";
        SqlCommand command = new SqlCommand(query, connection);
        command.Parameters.AddWithValue("@CategoryID", _selectedCategoryId);
        command.ExecuteNonQuery();
    }
    MessageBox.Show("Category deleted successfully.", "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information);
}
catch (SqlException sqlEx)
{
    if (sqlEx.Number == 547) // Foreign key violation
    {
        MessageBox.Show("Unable to delete this category because one or more products are still
using it.\n" +
                        "Please remove or reassign those products first.",
                        "Delete Not Allowed", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}
```

Screenshot evidence of the Category Management interface with delete capability:

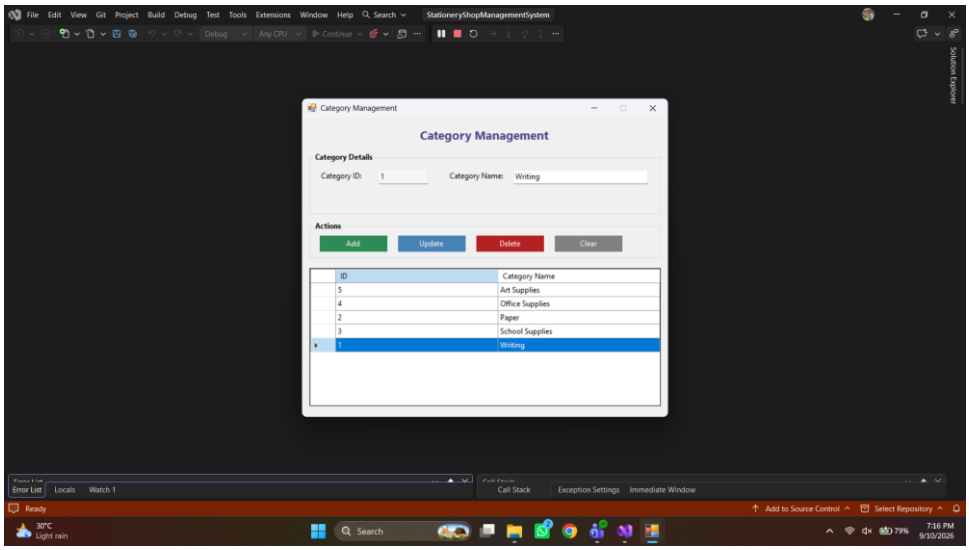


Fig 14: *Delete Operation Interface — Category Form with safety confirmation and FK error handling*

8. Marketing Purpose / Cost Implementation

To evaluate the real-world practicality of the Stationery Shop Management System, this chapter examines commercial demand, target users, and estimated cost implementation for a small retail stationery business.

8.1 Market Need

Small stationery shops often face challenges when managing their daily inventory and supplier records. Many of these shops still use manual paper notebooks, which makes it hard to track stock levels, find supplier contact numbers quickly, and notice when items are running out. A simple local desktop application provides an easy and practical solution for small shopkeepers to keep their product, category, and supplier records organized without needing complicated software or continuous internet access.

8.2 Target Customers

The target user base includes:

- **Independent Campus Stationery Shops:** Bookstores and stationery counters on and around university campuses such as AIUB.
- **School & College Supplies Vendors:** Neighborhood retailers selling notebooks, pens, pencils, geometry boxes, and art materials.
- **Office Stationery Supply Counters:** Small distributors supplying printing paper reams, staplers, files, and pens to corporate offices.

8.3 Estimated Cost Implementation

Because this project was developed as an academic university project, the following figures represent realistic commercial cost approximations if customized and deployed for a small local business:

Cost Component	Description	Estimated Cost (BDT)
Software Requirements & Design	Initial workflow analysis, database schema planning, and form layouts	10,000 BDT
Development & Database Setup	C# WinForms coding, SQL Server table creation, CRUD and validation logic	20,000 BDT
Deployment & Staff Training	Installing SQL Server Express on store PC, initial inventory data entry, staff	5,000 BDT

	onboarding	
Total One-Time Development Cost	**Complete desktop software solution delivered on store premises**	**35,000 BDT (~$300 USD)**

Hosting & Maintenance Cost: Because the current system runs locally on the shop computer, it does not require a separate cloud hosting service. Annual maintenance, periodic database backup assistance, and minor feature updates are estimated at approximately 5,000 to 10,000 BDT per year.

9. Conclusion

9.1 Summary & Key Learnings

The Stationery Shop Management System successfully demonstrates the practical application of Object-Oriented Programming (OOP) concepts in C# combined with relational database integration in Microsoft SQL Server Express. Through this project, our group gained hands-on experience designing graphical desktop user interfaces with Windows Forms, structuring business logic into encapsulated classes, and executing parameterized SQL commands through ADO.NET.

The project provided valuable practical reinforcement of key OOP 2 topics:

- **Encapsulation:** Implemented across Category, Supplier, and Product model classes by hiding private member fields (_productId, _unitPrice, etc.) behind public properties with controlled get/set accessors.
- **Constructors & Overloading:** Utilizing both default constructors (for form binding) and parameterized constructors (for rapid instantiation from database queries).
- **Method Implementation:** Creating domain methods such as IsLowStock() in the Product class to encapsulate business rules directly inside entity objects.
- **Robust Exception Handling:** Protecting database operations with structured try-catch-finally blocks, preventing crashes and gracefully handling foreign key constraints (SqlException error 547).
- **Event-Driven Programming:** Handling user interactions via button clicks, text box changes, and DataGridView row clicks to create an interactive desktop application.

9.2 Future Improvements

While the current application successfully fulfills all foundational requirements for inventory and supplier management, several realistic enhancements could be added in future versions:

- **Point-of-Sale (POS) & Billing Module:** Adding a customer checkout window to generate invoices, calculate sales tax, apply discounts, and print thermal receipts.
- **Barcode Scanner Integration:** Connecting a USB barcode scanner to instantly look up stationery items during fast-paced sales counter transactions.
- **Multi-User Role-Based Access Control:** Implementing separate user roles for 'Admin' (full CRUD, reporting, supplier management) and 'Cashier' (sales billing and search only).
- **Automated Backup & Cloud Synchronization:** Adding scheduled automated database backups to local external drives or cloud storage (Google Drive / OneDrive) to prevent data loss in case of hardware failure.

10. Reference

- **Microsoft C# Documentation:** Official technical documentation for C# programming language, language syntax, OOP constructs, and .NET Framework libraries.
  URL: https://learn.microsoft.com/en-us/dotnet/csharp/
- **Microsoft Windows Forms Documentation:** Official guide for developing desktop client applications, form controls, events, and DataGridView components.
  URL: https://learn.microsoft.com/en-us/dotnet/desktop/winforms/
- **Microsoft SQL Server Technical Documentation:** Official reference for Microsoft SQL Server Express, Transact-SQL (T-SQL) syntax, constraints, and relational database management.
  URL: https://learn.microsoft.com/en-us/sql/sql-server/
- **Microsoft ADO.NET Architecture & Overview:** Documentation on System.Data.SqlClient, SqlConnection, SqlCommand, SqlDataAdapter, and parameterized queries.
  URL: https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview
- **Microsoft Visual Studio Documentation:** Official guide for the Visual Studio Integrated Development Environment, Windows Forms designer, and debugging tools.
  URL: https://learn.microsoft.com/en-us/visualstudio/